

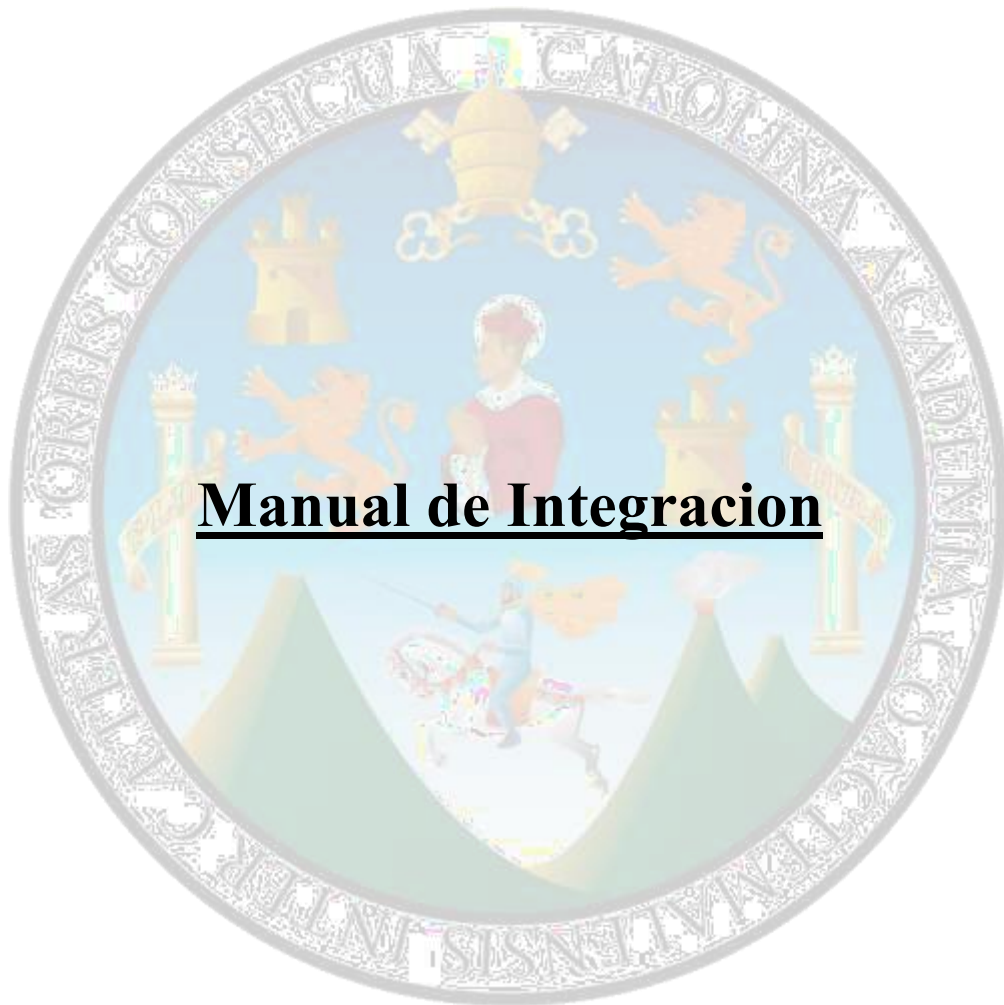
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2

CATEDRÁTICO: ING. ALVARO OBRAYAN HERNANDEZ GARCIA

TUTOR ACADÉMICO: LUIS ENRIQUE GARCIA GUTIERREZ



Josué Fuentes -202300668
Ariel Galicia -202300574
Randall Garcia -202202123
SECCIÓN: B

GUATEMALA, 03 DE SEPTIEMBRE DEL 2,025

ÍNDICE

ÍNDICE

1

OBJETIVOS DEL SISTEMA

GENERAL

Error! Bookmark not defined.

ESPECÍFICOS

Error! Bookmark not defined.

Error! Bookmark not defined.

INTRODUCCIÓN

Error! Bookmark not defined.

FLUJO DE LAS FUNCIONALIDADES DEL SISTEMA

Error! Bookmark not defined.

defined.

1. Acceso al sistema
 - 1.1. Inicio de sesión
 - 1.2. Crear nueva cuenta
2. Funcionalidades de usuario normal
 - 2.1. Gestión de correos
 - 2.2. Envío de correos
 - 2.3. Programación de correos
 - 2.4. Gestión de contactos
 - 2.5. Gestión de papelera
 - 2.6. Gestión de perfil
 - 2.7. Reportes personales
3. Funcionalidades del admin (root)
 - 3.1. Gestión de usuarios
 - 3.2. Gestión de comunidades
 - 3.3. Reportes globales del sistema
4. Navegación y controles generales
 - 4.1. Navegación entre ventanas
 - 4.2. Validación del sistema
 - 4.3. Mensajes del sistema
 - 4.4. Archivos de salida

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Error! Bookmark not defined.

Estructura base

Unit UComunidades (Josué Fuentes):

```
unit UComunidades;

{$mode ObjFPC}{$H+}

interface

uses
  SysUtils, Classes;

type
  PUsuarioComunidad = ^TUsuarioComunidad;
  TUsuarioComunidad = record
    id: String;           // neutral: cada quien convierte su tipo a String
    nombre: String;      // nombre o username visible
    siguiente: PUsuarioComunidad;
  end;

  PComunidad = ^TComunidad;
  TComunidad = record
    nombre: String;      // nombre de la comunidad
    usuarios: PUsuarioComunidad; // lista simple de usuarios de la comunidad
    siguiente: PComunidad; // siguiente comunidad (lista simple)
  end;

var
  ComunidadesHead: PComunidad = nil;

procedure InitComunidades;
function BuscarComunidad(const ANombre: String): PComunidad;
function CrearComunidad(const ANombre: String): PComunidad; // idempotente
function ExisteUsuarioEnComunidad(const C: PComunidad; const AId: String): Boolean;
function AgregarUsuarioAComunidad(const C: PComunidad; const AId, ANombre: String): Boolean;
function AgregarUsuarioAComunidadPorNombre(const ANombreComunidad, AId, ANombre: String): Boolean;
function EliminarUsuarioDeComunidad(const C: PComunidad; const AId: String): Boolean;
procedure ExportarReporteComunidadesDOT(const CarpetaDestino: String; const NombreArchivo: String = 'Reporte_Comunidades.dot');
procedure LiberarComunidades;

implementation

procedure InitComunidades;
begin
  ComunidadesHead := nil;
end;

function BuscarComunidad(const ANombre: String): PComunidad;
var
  p: PComunidad;
begin
  p := ComunidadesHead;
  while p <> nil do
    begin
      if SameText(p^.nombre, ANombre) then exit(p);
      p := p^.siguiente;
    end;
  Result := nil;
end;

function CrearComunidad(const ANombre: String): PComunidad;
var
  nueva: PComunidad;
begin
  Result := BuscarComunidad(ANombre);
  if Result <> nil then Exit; // ya existe
  New(nueva);
  nueva^.nombre := ANombre;
  nueva^.usuarios := nil;
  nueva^.siguiente := ComunidadesHead;
  ComunidadesHead := nueva;
  Result := nueva;
end;
```

Unit UComunidadesAdapters (Ariel Galicia):

```
unit UComunidadesAdapters;

{$mode ObjFPC}{$H+}

interface

uses
  SysUtils, Process, UComunidades;

function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;

// Intentar generar PNG desde .dot con Graphviz (opcional)
function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;

implementation

function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
begin
  Result := AgregarUsuarioAComunidadPorNombre(Comunidad, IntToStr(UserIdInt), UserName);
end;

function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;
begin
  Result := AgregarUsuarioAComunidadPorNombre(Comunidad, UserIdStr, UserName);
end;

function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;
var
  AProcess: TProcess;
begin
  Result := False;
  if not FileExists(RutaDOT) then Exit;
  AProcess := TProcess.Create(nil);
  try
    AProcess.Executable := 'dot'; // requiere Graphviz instalado
    AProcess.Parameters.Add('-Tpng');
    AProcess.Parameters.Add(RutaDOT);
    AProcess.Parameters.Add('-o');
    AProcess.Parameters.Add(RutaPNG);
    AProcess.Options := [poWaitOnExit];
    try
      AProcess.Execute;
      Result := FileExists(RutaPNG);
    except
      Result := False;
    end;
  finally
    AProcess.Free;
  end;
end;

end.
```

Form frmComunidades (Randall Garcia):

frmComunidades

Comunidades

Nombre:

Comunidad:

Correo:

```
{ TfrmComunidades }  
TfrmComunidades = class(TForm)  
    btnCrear: TButton;  
    btnAgregar: TButton;  
    btnSalir: TButton;  
    edNombreComunidad: TEdit;  
    edComunidad: TEdit;  
    edCorreo: TEdit;  
    lblNombre1: TLabel;  
    lblTitulo: TLabel;  
    lblNombre: TLabel;  
    lblComunidad: TLabel;  
    lblCorreo: TLabel;  
    procedure btnCrearClick(Sender: TObject);  
    procedure btnAgregarClick(Sender: TObject);  
    procedure btnSalirClick(Sender: TObject);  
    procedure FormCreate(Sender: TObject);  
    procedure FormShow(Sender: TObject);  
private  
    function BuscarNombrePorCorreo(const Email: String): String;  
    procedure Msg(const s: String);  
public  
end;
```

Implementaciones a proyecto

```

unit UComunidades;

{$mode ObjFPC}{$SH+}

interface

uses
  SysUtils, Classes;

type
  PUsuarioComunidad = ^TUsuarioComunidad;
  TUsuarioComunidad = record
    id: String;           // neutral: cada quien convierte su tipo a String
    nombre: String;       // nombre o username visible
    siguiente: PUsuarioComunidad;
  end;

  PComunidad = ^TComunidad;
  TComunidad = record
    nombre: String;       // nombre de la comunidad
    usuarios: PUsuarioComunidad; // lista simple de usuarios de la comunidad
    siguiente: PComunidad; // siguiente comunidad (lista simple)
  end;

var
  ComunidadesHead: PComunidad = nil;

procedure InitComunidades;
function BuscarComunidad(const ANombre: String): PComunidad;
function CrearComunidad(const ANombre: String): PComunidad; // idempotente
function ExisteUsuarioEnComunidad(const C: PComunidad; const AId: String): Boolean;
function AgregarUsuarioAComunidad(const C: PComunidad; const AId, ANombre: String): Boolean;
function AgregarUsuarioAComunidadPorNombre(const ANombreComunidad, AId, ANombre: String): Boolean;
function EliminarUsuarioDeComunidad(const C: PComunidad; const AId: String): Boolean;
procedure ExportarReporteComunidadesDOT(const CarpetaDestino: String; const NombreArchivo: String = 'Reporte_Comunidades.dot');
procedure LiberarComunidades;

implementation

procedure InitComunidades;
begin
  ComunidadesHead := nil;
end;

function BuscarComunidad(const ANombre: String): PComunidad;
var
  p: PComunidad;
begin
  p := ComunidadesHead;
  while p <> nil do
    begin
      if SameText(p^.nombre, ANombre) then exit(p);
      p := p^.siguiente;
    end;
  Result := nil;
end;

function CrearComunidad(const ANombre: String): PComunidad;
var
  nueva: PComunidad;
begin
  Result := BuscarComunidad(ANombre);
  if Result <> nil then Exit; // ya existe
  New(nueva);
  nueva^.nombre := ANombre;
  nueva^.usuarios := nil;
  nueva^.siguiente := ComunidadesHead;
  ComunidadesHead := nueva;
  Result := nueva;
end;

```

```

var
  u: PUsuarioComunidad;
begin
  Result := False;
  if C = nil then Exit;
  u := C^.usuarios;
  while u <> nil do
    begin
      if SameText(u^.id, AId) then exit(True);
      u := u^.siguiente;
    end;
  end;
end;

function AgregarUsuarioAComunidad(const C: PComunidad; const AId, ANombre: String): Boolean;
var
  nuevo: PUsuarioComunidad;
begin
  Result := False;
  if (C = nil) or (AId = '') then Exit;
  if ExisteUsuarioEnComunidad(C, AId) then Exit; // no duplicar
  New(nuevo);
  nuevo^.id := AId;
  nuevo^.nombre := ANombre;
  nuevo^.siguiente := C^.usuarios;
  C^.usuarios := nuevo;
  Result := True;
end;

function AgregarUsuarioAComunidadPorNombre(const ANombreComunidad, AId, ANombre: String): Boolean;
var
  c: PComunidad;
begin
  c := BuscarComunidad(ANombreComunidad);
  if c = nil then c := CrearComunidad(ANombreComunidad);
  Result := AgregarUsuarioAComunidad(c, AId, ANombre);
end;

function EliminarUsuarioDeComunidad(const C: PComunidad; const AId: String): Boolean;
var
  ant, act: PUsuarioComunidad;
begin
  Result := False;
  if C = nil then Exit;
  ant := nil; act := C^.usuarios;
  while act <> nil do
    begin
      if SameText(act^.id, AId) then
        begin
          if ant = nil then C^.usuarios := act^.siguiente
          else ant^.siguiente := act^.siguiente;
          Dispose(act);
          Exit(True);
        end;
      ant := act;
      act := act^.siguiente;
    end;
  end;
end;

procedure ExportarReporteComunidadesDOT(const CarpetaDestino: String; const NombreArchivo: String);
var
  dot: TStringList;
  c: PComunidad;
  u: PUsuarioComunidad;
  rutaDot, prevComId, thisComId, firstUserId, prevUserId: String;
  idxC, idxU: Integer;
  communitiesRankLine: String;

function Esc(const s: String): String;
begin
  Result := StringReplace(s, '"', '\\"', [rfReplaceAll]);
end;

procedure EnsureDir(const Dir: String);
begin
  if (Dir <> '') and (not DirectoryExists(Dir)) then
    ForceDirectories(Dir);
end;

```

```

begin
  dot := TStringList.Create;
  try
    dot.Add('digraph "Reporte de Comunidades" {');
    dot.Add('  rankdir=LR;');
    dot.Add('  labelloc="t";');
    dot.Add('  label="Reporte de Comunidades";');
    dot.Add('  node [fontname="Helvetica"];');
    dot.Add('  edge [arrowsize=0.8];');

    // estilos sugeridos (como en tu mockup)
    dot.Add(' // Estilos');
    dot.Add(' subgraph cluster_legend { style=invis; }');
    dot.Add(' // Comunidad: caja azul');
    dot.Add(' // Usuario: caja amarilla');

    idxC := 0;
    prevComId := '';
    communitiesRankLine := ' { rank=same; ';

    c := ComunidadesHead;
    while c <> nil do
      begin
        Inc(idxC);
        thisComId := Format('c%d', [idxC]);

        // Nodo de comunidad (fila superior)
        dot.Add(Format(' %s [shape=box, style="rounded,filled", fillcolor="#a9d1e6", color="#2c3e50", ' +
          'label=<b>%s</b>>, width=2.8, height=0.9];',
          [thisComId, Esc(c^.nombre)]));

        // Conexión comunidad anterior -> comunidad actual (lista simple)
        if prevComId <> '' then
          dot.Add(Format(' %s -> %s;', [prevComId, thisComId]));
        prevComId := thisComId;

        // Mantener en la misma fila
        communitiesRankLine := communitiesRankLine + thisComId + ' ';

        // Usuarios debajo (columna vertical)
        u := c^.usuarios;
        idxU := 0;
        firstUserId := '';
        prevUserId := '';

        while u <> nil do
          begin
            Inc(idxU);
            // id único por comunidad/usuario
            // nota: usamos índice para evitar choques por emails con caracteres especiales
            dot.Add(Format(' %s_u%d [shape=box, style="rounded,filled", fillcolor="#f6f4c0", color="#7f8c8d", ' +
              'label=< %s >, width=3.3, height=0.8];',
              [thisComId, idxU, Esc(u^.id)]));

            if idxU = 1 then
              begin
                firstUserId := Format('%s_u%d', [thisComId, idxU]);
                // Flecha comunidad -> primer usuario (vertical)
                dot.Add(Format(' %s -> %s;', [thisComId, firstUserId]));
              end
            else
              begin
                // Flechas verticales usuarioN-1 -> usuarioN
                dot.Add(Format(' %s -> %s;', [prevUserId, Format('%s_u%d', [thisComId, idxU])]));
              end;

            prevUserId := Format('%s_u%d', [thisComId, idxU]);
            u := u^.siguiente;
          end;
        end;
      end;
    end;
  end;
end;

```

```

1  unit UComunidadesAdapters;
.
.
.  {Snode ObjFPC}{SH+}
.
5  interface
.
.  uses
.    SysUtils, Process, UComunidades;
.
10 function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
. function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;
.
.  // Intentar generar PNG desde .dot con Graphviz (opcional)
.  function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;
15
.  implementation
.
.  function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
.  begin
20      Result := AgregarUsuarioAComunidadPorNombre(Comunidad, IntToStr(UserIdInt), UserName);
.  end;
.
.  function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;
.  begin
25      Result := AgregarUsuarioAComunidadPorNombre(Comunidad, UserIdStr, UserName);
.  end;
.
.  function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;
.  var
.  begin
30      AProcess: TProcess;
.  begin
.      Result := False;
.      if not FileExists(RutaDOT) then Exit;
.      AProcess := TProcess.Create(nil);
.
35      try
.          AProcess.Executable := 'dot'; // requiere Graphviz instalado
.          AProcess.Parameters.Add('-Tpng');
.          AProcess.Parameters.Add(RutaDOT);
.          AProcess.Parameters.Add('-o');
40          AProcess.Parameters.Add(RutaPNG);
.          AProcess.Options := [pmWaitOnExit];
.          try
.              AProcess.Execute;
.              Result := FileExists(RutaPNG);
45          except
.              Result := False;
.          end;
.          finally
.              AProcess.Free;
50          end;
.      end;
.
53 end;

```

```

1  unit UComunidadesAdapters;
.
.
.  {Snode ObjFPC}{SH+}
.
5  interface
.
.  uses
.    SysUtils, Process, UComunidades;
.
10 function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
. function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;
.
.  // Intentar generar PNG desde .dot con Graphviz (opcional)
.  function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;
15
.  implementation
.
.  function AddUserToCommunity_IntId(const Comunidad: String; const UserIdInt: Integer; const UserName: String): Boolean;
.  begin
20      Result := AgregarUsuarioAComunidadPorNombre(Comunidad, IntToStr(UserIdInt), UserName);
.  end;
.
.  function AddUserToCommunity_StrId(const Comunidad, UserIdStr, UserName: String): Boolean;
.  begin
25      Result := AgregarUsuarioAComunidadPorNombre(Comunidad, UserIdStr, UserName);
.  end;
.
.  function TryGeneratePNGFromDOT(const RutaDOT, RutaPNG: String): Boolean;
.  var
.  begin
30      AProcess: TProcess;
.  begin
.      Result := False;
.      if not FileExists(RutaDOT) then Exit;
.      AProcess := TProcess.Create(nil);
.
35      try
.          AProcess.Executable := 'dot'; // requiere Graphviz instalado
.          AProcess.Parameters.Add('-Tpng');
.          AProcess.Parameters.Add(RutaDOT);
.          AProcess.Parameters.Add('-o');
40          AProcess.Parameters.Add(RutaPNG);
.          AProcess.Options := [pmWaitOnExit];
.          try
.              AProcess.Execute;
.              Result := FileExists(RutaPNG);
45          except
.              Result := False;
.          end;
.          finally
.              AProcess.Free;
50          end;
.      end;
.
53 end;

```

```

unit fComunidades;

($mode ObjFPC)($H+)

interface

uses
  Classes, SysUtils, Forms, Controls, StdCtrls, Dialogs,
  UComunidades, UComunidadesAdapters, uUsers, fMain;

type
  ( TfrmComunidades )
  TfrmComunidades = class(TForm)
    btnCrear: TButton;
    btnAgregar: TButton;
    btnSalir: TButton;
    edNombreComunidad: TEdit;
    edComunidad: TEdit;
    edCorreo: TEdit;
    lblNombre1: TLabel;
    lblTitulo: TLabel;
    lblNombre: TLabel;
    lblComunidad: TLabel;
    lblCorreo: TLabel;
    procedure btnCrearClick(Sender: TObject);
    procedure btnAgregarClick(Sender: TObject);
    procedure btnSalirClick(Sender: TObject);
    procedure FormCreate(Sender: TObject);
    procedure FormShow(Sender: TObject);
  private
    function BuscarNombrePorCorreo(const Email: String): String;
    procedure Msg(const s: String);
  public
  end;

var
  frmComunidades: TfrmComunidades;

implementation

($R *.ifm)

const
  OUT_DIR = "Root-Reportes";

procedure TfrmComunidades.Msg(const s: String);
begin
  ShowMessage(s);
end;

// Intenta obtener el nombre desde tu lista de usuarios por email.
// Variante A: si uUsers tiene una función para buscar por email, úsala aquí.
// Variante B: si no la tienes, de momento devuelve el mismo correo como "nombre" (fallback).
function TfrmComunidades.BuscarNombrePorCorreo(const Email: String): String;
var
  U: PUser;
begin
  // Tu uUsers ya trae FindUserByEmailOrUsername (acepta email o username)
  U := FindUserByEmailOrUsername(Email);
  if U <> nil then
    Result := U^.Name // nombre real del usuario
  else
    Result := ''; // no encontrado
  end;
end;

procedure TfrmComunidades.FormShow(Sender: TObject);
begin
  // Limpia campos cada vez que abras el form
  edNombreComunidad.Clear;
  edComunidad.Clear;
  edCorreo.Clear;
end;

```

```

procedure TfrmComunidades.btnAgregarClick(Sender: TObject);
var
  con, email, nombre: String;
  U: PUser;
begin
  con := Trin(edComunidad.Text);
  email := Trin(edCorreo.Text);

  if (con = '') or (email = '') then
  begin
    Msg('Completa Comunidad y Correo. ');
    Exit;
  end;

  // Verificar que el correo exista en tu lista de usuarios
  U := FindUserByEmailOrUsername(email);
  if U = nil then
  begin
    Msg('El correo no existe en la lista de usuarios: ' + email);
    Exit;
  end;

  nombre := U^.Name; // ya que existe, tomamos el nombre real

  // Crea la comunidad si no existe
  if BuscarComunidad(con) = nil then
    CrearComunidad(con);

  // En comunidades usamos el correo como id String (neutral)
  if AddUserToCommunity_StrId(con, email, nombre) then
    Msg('Usuario agregado a ' + con + ' .')
  else
    Msg('No se pudo agregar');
  end;
end;

procedure TfrmComunidades.btnSalirClick(Sender: TObject);
begin
  frmComunidades.Hide;
  frmRoot.Show;
end;

procedure TfrmComunidades.FormCreate(Sender: TObject);
begin
  1
end;

end.

```